

TRABAJO PRÁCTICO INTEGRADOR:**PROYECTO: GESTIÓN DE DATOS DE PAÍSES**

Alumnos	Gualla Mariano, Furfaro Ivan
Carrera	Tecnicatura Universitaria en Programación
Materia	Programación 1
Comisión	21
Grupo	33
Docente Titular	Alberto Cortez
Docentes Adjuntos	Ariel Enferrel, Martín A. García, Cinthia Rigoni
Tutor de comisión	Neyén Bianchi
Fecha de entrega	Junio de 2026

TABLA DE CONTENIDO

1 - Marco Teórico.....	3
2 - Decisiones técnicas y Arquitectura	5
3 - Flujo de operaciones principales.	6
4 - Capturas de pantalla de la ejecución	7
5 - Dificultades y conclusiones.....	9
6 - Bibliografía / Webgrafía	9
7 - Links	10

1 – MARCO TEÓRICO

LISTAS

Las listas fueron utilizadas para almacenar conjuntos de datos que debían recorrerse mediante iteraciones.

Su primer uso fue almacenar los continentes válidos admitidos por el sistema.

Esta lista permitió validar la entrada del usuario al momento de registrar o filtrar países, verificando que el continente ingresado coincidiera con alguno de los elementos almacenados.

También se utilizaron listas para almacenar los registros obtenidos del archivo CSV. Cada registro representa un país y se almacena como un diccionario dentro de una lista. Esto permitió recorrer los datos mediante estructuras repetitivas para realizar búsquedas, filtrados, ordenamientos y cálculos estadísticos.

Durante las funcionalidades de filtrado por población y superficie, se recorrieron las listas de registros para seleccionar únicamente aquellos países cuyos valores se encontraban dentro de los rangos especificados por el usuario.

DICCIONARIOS

Los diccionarios fueron utilizados para representar cada país almacenado en el archivo CSV. Mediante el dictReader, cada fila se convirtió en un diccionario donde los encabezados actúan como claves y los datos de cada país como valores asociados.

Esta estructura permitió acceder a la información utilizando el nombre de cada campo.

Por ejemplo, fue posible consultar o modificar datos específicos mediante claves como "nombre", "población" o "continente" sin necesidad de conocer la posición de los datos dentro del registro, si por algún motivo el orden de los encabezados se modifica, la ejecución seguiría su curso sin inconvenientes.

Su uso facilitó las tareas de búsqueda, filtrado, actualización y estadísticas a partir de la información almacenada.

FUNCIONES

El programa fue desarrollado siguiendo un diseño modular basado en funciones. Cada función cumple una responsabilidad específica y encapsula una tarea concreta dentro del sistema.

Este enfoque permitió dividir el problema en partes más pequeñas y fáciles de mantener. Por ejemplo, existen funciones dedicadas a validar entradas, leer archivos, agregar países, actualizar información, realizar búsquedas, ordenar registros y calcular estadísticas.

Además de mejorar la organización del código, las funciones favorecen la reutilización, ya que una misma función puede ser invocada desde distintos puntos del programa sin necesidad de duplicar instrucciones. Y la corrección de errores se ve facilitada por que estos suceden únicamente dentro de su estructura.

CONDICIONALES

Las estructuras condicionales fueron utilizadas de forma constante para controlar el flujo de ejecución del programa y permitir la toma de decisiones según distintas situaciones.

Su aplicación incluyó la validación de datos ingresados por el usuario, la verificación de la existencia del archivo CSV, la comprobación de registros almacenados y el control de errores durante diversas operaciones.

También fueron empleadas para gestionar la navegación entre las distintas funcionalidades del sistema, permitiendo ejecutar acciones específicas según la opción seleccionada por el usuario. Gracias a ello, el programa puede adaptarse a diferentes escenarios y responder adecuadamente ante entradas inválidas.

ORDENAMIENTO

Para implementar la funcionalidad de ordenamiento de datos se utilizó el algoritmo de ordenamiento burbuja.

Si bien Python dispone de herramientas más eficientes como `sort` o `sorted`, los requerimientos del trabajo práctico limitaron el uso de funciones predefinidas para este proceso.

Por este motivo se optó por una implementación manual del algoritmo.

El método burbuja consiste en recorrer repetidamente una colección de datos, comparando elementos consecutivos e intercambiando sus posiciones cuando no se encuentran en el orden deseado.

Este procedimiento se repite hasta que todos los elementos quedan correctamente ordenados.

Aunque su rendimiento es inferior al de otros algoritmos de ordenamiento, su implementación permitió cumplir los requisitos del proyecto y comprender el funcionamiento interno de un proceso de ordenación.

ESTADÍSTICAS BÁSICAS

El sistema incorpora una funcionalidad de estadísticas que permite obtener información general a partir de los datos almacenados en el archivo CSV.

Se recorren los registros válidos y se realizan operaciones aritméticas sobre los campos numéricos. Entre las estadísticas implementadas se encuentran el cálculo del promedio de población mundial, el promedio de superficie territorial y la identificación de los países con mayor y menor cantidad de habitantes.

También, se contabiliza la cantidad de países registrados por continente. Durante estos procesos se validan los datos obtenidos del archivo, ignorando aquellos registros que contengan información incompleta o inválida para evitar resultados incorrectos.

Estas operaciones permiten transformar los datos almacenados en información útil para el análisis y la consulta.

ARCHIVO CSV

El archivo fue construido a partir de datasets públicos sobre países del mundo y posteriormente adaptado para ajustarse a los requerimientos del proyecto. Entre las modificaciones realizadas se encuentran la conservación exclusiva de las columnas nombre, población, superficie y continente, la conversión de superficies expresadas en millas cuadradas a kilómetros cuadrados y la normalización de los nombres de los continentes a las categorías: Africa, America, Asia, Europa, Oceania y Antartida. Durante la ejecución del programa, el archivo CSV actúa como almacenamiento persistente de la información. Su lectura se realiza mediante el DictReader, mientras que la escritura y actualización de registros se efectúan mediante writer y DictWriter.

La utilización de este formato permitió mantener los datos organizados, facilitar su manipulación mediante diccionarios y conservar la información entre distintas ejecuciones del programa.

2 – DECISIONES TÉCNICAS Y ARQUITECTURA

A) Modularización mediante funciones

Se decidió estructurar el programa de forma modular utilizando funciones. Cada una cumple una responsabilidad específica y puede ser invocada desde la función principal o desde otras funciones según sea necesario. Con el objetivo de reducir la duplicación de código, mejorar la organización del sistema y facilitar la detección y corrección de errores.

B) Validación de datos de entrada

Se implementaron funciones de validación específicas para cada tipo de dato requerido. Esto permitió garantizar la integridad de la información almacenada, evitando que el usuario ingresara valores incompatibles con los requerimientos de cada funcionalidad.

C) Implementación de algoritmos de ordenamiento manuales

Para las funcionalidades de ordenamiento se implementó el algoritmo de ordenamiento burbuja. Esta decisión fue tomada para cumplir con las restricciones establecidas por el trabajo práctico.

D) Uso controlado del manejo de excepciones

Se decidió utilizar estructuras try/except únicamente en situaciones donde podían producirse errores impredecibles, principalmente durante la manipulación de archivos y la conversión de datos. El resto de las validaciones fueron realizadas mediante lógica de programación y estructuras condicionales.

E) Organización de la función principal

La función main fue diseñada como punto central de control del programa. Su responsabilidad principal consiste en mostrar el menú de opciones, gestionar el flujo general de ejecución y proporcionar a las distintas funcionalidades la lista de continentes utilizada para las validaciones correspondientes.

F) Control de ejecución mediante if __name__ == "__main__"

Se incorporó la estructura if __name__ == "__main__": para garantizar que el programa principal solo se ejecute cuando el archivo sea utilizado como punto de entrada de la aplicación. Esto evita ejecuciones no deseadas en caso de que el módulo sea importado desde otro archivo.

G) Mostrar resultados

Se incorporó la función detener_ejecucion() que permite ver los resultados de cada operación y continuar cuando el usuario ingrese un tecla

3 - FLUJO DE OPERACIONES PRINCIPALES

```
Inicio
|
Crear archivo si no existe
|
Mostrar menú
|
Opción seleccionada
|
- Agregar país
|   |
|   |__ Validar datos
|   |__ Verificar duplicados
|   |__ Guardar registro
|
- Actualizar país
|   |
|   |__ Buscar país
|   |__ Modificar datos
|   |__ Reescribir archivo
|
- Buscar país
|   |
|   |__ Mostrar resultados
|
- Filtrar datos
|   |
|   |__ Mostrar coincidencias
|
- Ordenar datos
|   |
|   |__ Mostrar ordenamiento
|
- Estadísticas
|   |
|   |__ Calcular indicadores
|
- Salir
|
Fin del programa
```

4 - Capturas de pantalla de la ejecución

```
=== Gestión de Datos de Países ===
1 - Agregar país
2 - Actualizar datos de país
3 - Buscar país
4 - Filtrar por datos
5 - Ordenar país
6 - Estadísticas
0 - Salir

-> 1
=== Agregar País ===
Ingrese nombre del país: Argelia
Ingrese población del país: 47534765
Ingrese superficie en Km2 del país: 2381741

Africa | America | Asia | Europa | Oceania | Antartida |
Ingrese continente donde se encuentra el país: Africa

=== Nuevo país guardado correctamente ===

Presione una tecla para continuar...[]
```

Ilustración 1 Menú y Funcionalidad Agregar

```
-> 3
=== Buscar país ===
Seleccione un país: arg

=== País encontrado ===
Nombre: Argentina | Población actual: 39921833 | Superficie actual: 7166217 Km2 | Continente: America

=== País encontrado ===
Nombre: Argelia | Población actual: 47534765 | Superficie actual: 2381741 Km2 | Continente: Africa

Presione una tecla para continuar...[]
```

Ilustración 2 Búsqueda parcial de nombre

```
=== Filtrar países por continente ===  
Africa | America | Asia | Europa | Oceania | Antartida |  
Ingrese continente: oceania  
  
Países en Oceania:  
- American Samoa  
- Australia  
- Cook Islands  
- Fiji  
- French Polynesia  
- Guam  
- Kiribati  
- Marshall Islands  
- Micronesia, Fed. St.  
- Nauru  
- New Caledonia  
- New Zealand  
- N. Mariana Islands  
- Palau  
- Papua New Guinea  
- Samoa  
- Solomon Islands  
- Tonga  
- Tuvalu  
- Vanuatu  
- Wallis and Futuna  
  
Presione una tecla para continuar...[]
```

Ilustración 3 Filtrar por continente

```
-> 6  
  
=== Estadísticas ===  
  
País con menor población:  
St Pierre & Miquelon | Población: 7026  
  
País con mayor población:  
China | Población: 1313973713  
  
=== Promedio de población ===  
Mundial: 29503815  
  
=== Promedio de superficie ===  
Mundial: 1391557 km2  
  
=== Cantidad de países por continente ===  
Africa: 58  
Asia: 44  
Europa: 40  
America: 50  
Oceania: 21  
  
Presione una tecla para continuar...[]
```

Ilustración 4 Estadísticas

5 - DIFICULTADES Y CONCLUSIONES

Una de las primeras dificultades encontradas fue la obtención de un dataset adecuado para el proyecto. Debido a que no se encontró un archivo CSV que cumpliera exactamente con los requerimientos planteados, se decidió construir un dataset propio a partir de distintas fuentes públicas, realizando adaptaciones en los nombres de las columnas, unidades de medida y clasificación de continentes.

Otro desafío importante fue comprender el funcionamiento del DictReader y la estructura de datos generada a partir de la lectura del archivo. Fue necesario realizar diversas pruebas para entender cómo acceder correctamente a los registros y manipular la información almacenada.

También surgieron inconvenientes relacionados con registros incompletos o campos vacíos dentro del archivo CSV. Estos casos provocaban errores durante algunos cálculos y recorridos de datos. La solución adoptada consistió en validar la información antes de procesarla e ignorar aquellos registros que no contuvieran todos los datos necesarios.

Durante el desarrollo se observó además que algunas funciones acumulaban demasiadas responsabilidades, dificultando su mantenimiento y comprensión. Para resolver este problema se aplicó una mayor modularización, dividiendo ciertas tareas en funciones auxiliares más pequeñas y específicas.

Como conclusión, el proyecto permitió aplicar de manera práctica conceptos fundamentales de programación, como funciones, listas, diccionarios, archivos CSV, estructuras de control, validación de datos y manejo de excepciones.

También, nos puso en evidencia la importancia de la planificación, la depuración de errores y la organización modular del código para desarrollar aplicaciones más robustas y mantenibles.

6 - BIBLIOGRAFIA/WEBGRAFIA

Material de cátedra

Universidad Tecnológica Nacional (UTN).

Tecnicatura Universitaria en Programación a Distancia.
Programación I.

“Estructuras de Datos Complejos - Actividad III”.

“Archivos CSV (listas y diccionarios)”.

“Manejo de error con archivos – Actividad IV”.

Documentación oficial de Python.

Módulo csv - DictReader.

Disponible en:

<https://docs.python.org/es/3/library/csv.html#csv.DictReader>

Tipos de datos incorporados: diccionarios (dict).

Disponible en:

<https://docs.python.org/es/3/library/stdtypes.html#dict>

Fernando L. Countries of the World Dataset.

Kaggle.

Disponible en:

<https://www.kaggle.com/datasets/fernandol/countries-of-the-world>

DataHub.

Country List Dataset.

Disponible en:

<https://datahub.io/core/country-list>

7 - LINKS

Repositorio GitHub

https://github.com/MarianoGualla/Professional-Learning/tree/main/university/UTN/Programming%201/Integrative%20jobs/Gestion_de_datos

Video explicativo

https://www.youtube.com/watch?v=C_BKAdEIrw&t=6s